HeLSci · @data training · Introduction to LLMs

Introduction to LLMs

Day 1

Cédric GILON

Monday 9 November 2026 · Brachet AW1.203

ULB CONTINUING EDUCATION HELSCI HEALTH & LIFE SCIENCES

Who am I?

Cédric Gilon

Research background

- Master in science
- PhD in machine learning (2024): detection of heart disease

Today

- Machine Learning Engineer and coordinator of the ULB AI plan, at the ULB IT department
- Researcher and scientific collaborator at IRIDIA

Two days

Day 1	morning	• machine learning → LLM: tokens, context, training • sampling, temperature
	afternoon	• prompts, API • RAG: answer from your documents
Day 2	morning	• when RAG fails, structured outputs • evaluate the quality of the answers
	afternoon	• cost, data protection, when not to use an LLM • project

Objectives

2 days of training on LLMs, to:

1. know how an LLM works
2. use an LLM as a tool to work on data
3. know what it can do, and know its limits
4. evaluate the quality of its answers
5. build a small program that uses an LLM

Wooclap

- What do you expect from these two days?
- What do you know about LLMs?

Small project with an LLM: do you already have an idea from your field?

Exercises

- Suggestion: work in **pairs**
- **marimo** notebooks: change the data, run, observe
- The code to write is marked `# TODO`
- A check after each step:

  ✅ ok · ⏳ to do · ❌ not yet (with a hint if needed)

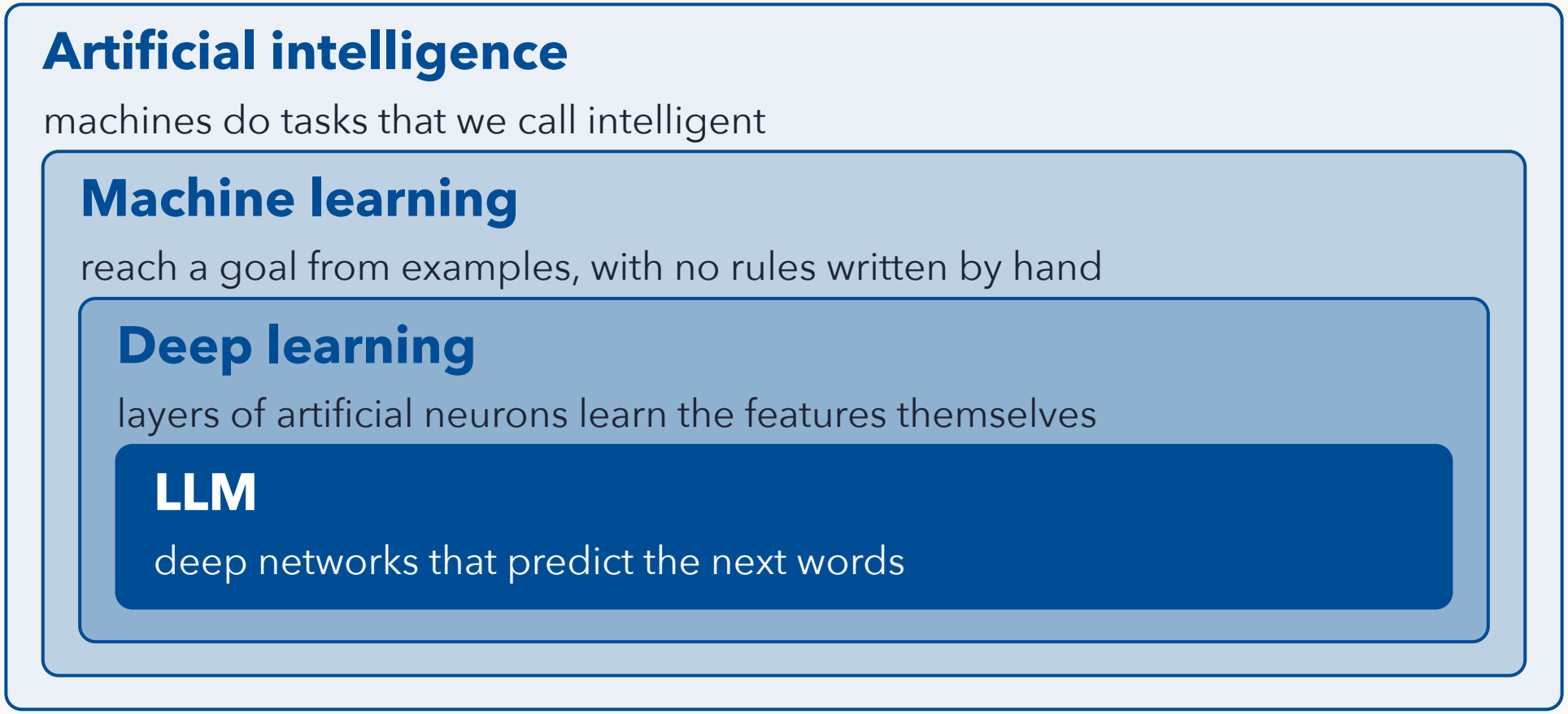
Check the installation

```
cd 00-install-check
uv run python install_check.py # READY?
```

- Ollama is running and `llama3.2:3b` is downloaded (exercise 2)
- Keys in `.env`:
  - Groq (with a **q**, not Grok) for the exercises today
  - Gemini for tomorrow
- `uv` and VS Code: already installed

From machine learning to LLMs

Four nested fields



Reminder: machine learning

- Learn from data: the developer chooses the features
- The loop you know:



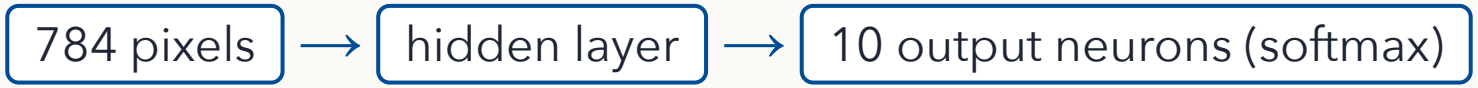
- The same rules apply to LLMs: train-test set, overfitting, data leakage

MNIST: who chooses the features?

Model	Accuracy	The features
Logistic regression	92–93%	the 784 pixels, as they are
MLP, 1–2 hidden layers	97–98%	learned by the hidden layer: strokes, curves
CNN	99.5%	learned by filters, layer after layer

The network learns its own features: less manual work, but more compute

From machine learning to deep learning



- Image or text: which features to choose? The network learns them itself
- With more layers: edges → shapes → object
- Biology: pixels → cell edges → nuclei → cell type

Summers and winters



1956–1974
the term "AI" (1956), the perceptron (1958)

1974–1980
limits of the perceptron, Lighthill report (1973)

1980–1987
expert systems, backpropagation (1986)

1987–1993
expert systems too expensive

1993–2012
ML: SVM, random forests, boosting

2012–2026
AlexNet (2012), Transformer (2017), ChatGPT (2022)

Promises, then disappointments. Funding follows the highs and the lows

Source: Rosenblatt, *Psychological Review* 65(6), 1958; Rumelhart et al., *Nature* 323, 1986; Cortes and Vapnik, *Machine Learning* 20, 1995; Breiman, *Machine Learning* 45, 2001; Krizhevsky et al., NeurIPS 2012

Deep learning: why now?

Why now?

- Data: the web (Wikipedia, StackOverflow), books
- Compute: GPUs (parallel computing)
- Libraries: PyTorch, TensorFlow

But ...

- More data, more compute
- A deep network is harder to explain than a decision tree
- With 1,000 examples, a logistic regression can beat a deep network

From deep learning to LLMs

Recognition of words or full sentences

- recurrent models (RNN)
- Transformers

- As input, one one-hot per word (see the ML module): one column per word, which is huge
- The solution: embeddings, short vectors that carry meaning

The next token: a multiclass classification

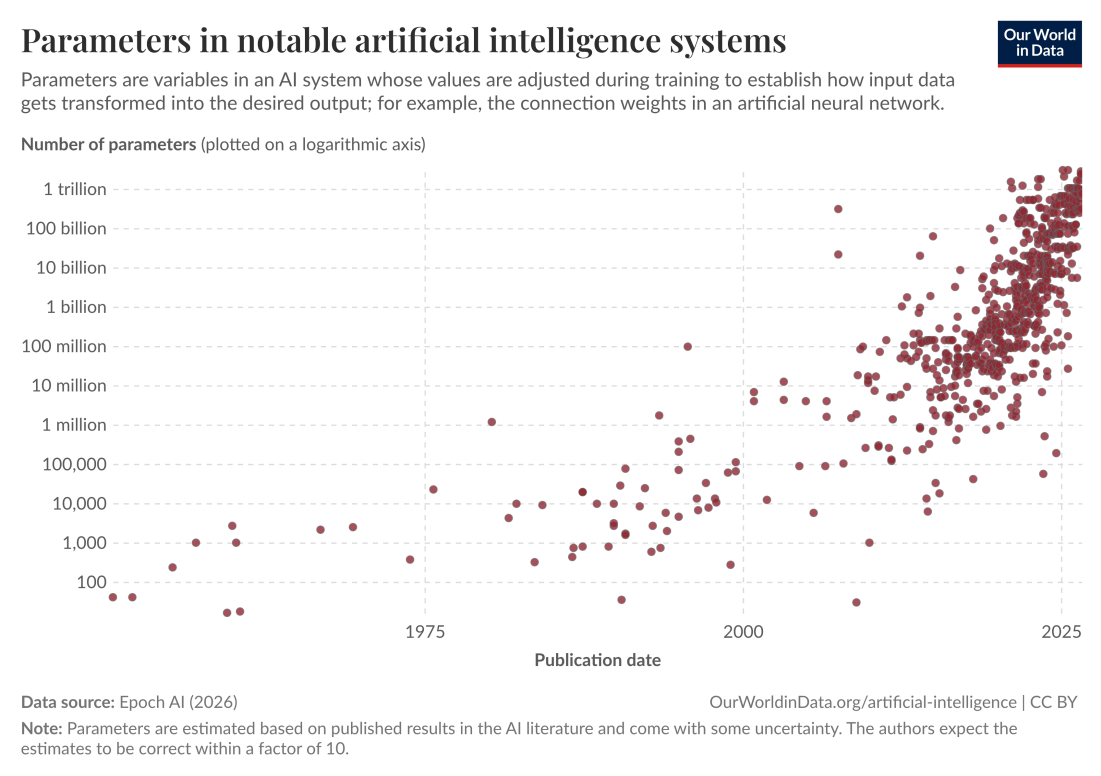
	MNIST (ML module)	**LLM**
Input	784 pixels	the previous tokens
Classes	10 digits	≈ 200,000 tokens (exercise 1)
Last layer	scores (logits) → softmax → probabilities	
Error	cross-entropy	
Labels	written by hand	the next token of the text

Free labels (self-supervised): all digital text (e.g. the web, books) can be training data

An LLM

A deep network trained to predict the next token, on a large part of the public web

Large: billions of **parameters**, trained on thousands of GPUs



Source: Our World in Data (CC BY), data Epoch AI (2026)

The GPU: thousands of computations at a time



- A GPU is made for images: each pixel is computed on its own, at the same time as the others
- A layer of neurons too: each neuron computes its weighted sum without waiting for the others
- CPU: a few cores (10 on this Mac). H100 GPU: 16,896 simpler cores
- AlexNet (2012): 5 to 6 days of training on 2 gaming GPUs (GTX 580)

Source: Krizhevsky et al., NeurIPS 2012

ML: often one task, one model

- Sentiment of a review: model 1
- Translation: model 2
- Names in a text: model 3

Each one is trained on its own labelled data

RNN (1990): one token after the other

Reads the tokens in order and passes a **memory** to the next step

Example: Twitter post ⟶ sentiment analysis

Two limits:
1. the memory fades on a long text (*long term dependency problem*)
2. step-by-step computing makes training slow

New architectures to remember better: LSTM (1997), then GRU (2014)

Source: Elman, *Cognitive Science* 14(2), 1990; Hochreiter and Schmidhuber, 1997; Cho et al., arXiv 2014

Attention (2014): look at all the words

In translation: before it writes each French word, the model looks at **all** the English words

A cute teddy bear is reading
⟶ Un ours en peluche **mignon** lit

To write *mignon*, the model must look mostly at *cute*: the two words are not at the same place in the sentence

Source: Bahdanau et al., arXiv 2014, ICLR 2015

Each word looks at all the others

RNN: the last word reaches the first one only through all the steps between them



Attention: direct links, each one weighted. The thicker the line, the more the word counts



The context chooses the meaning

- **it**?
  - The animal didn’t cross the street because **it** (the animal) was too *tired*
  - The animal didn’t cross the street because **it** (the street) was too *wide*
- **mole**? the animal, the mole (unit in chemistry) or a spot on the skin

With attention, the context sets the meaning

Attention in one formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

1. QK^T: question × label, one score per pair, high when the two vectors point in the same direction
2. softmax: the scores of a token become weights that sum to 1 ($\sqrt{d}$ only keeps the computation stable)
3. $\times V$: the contents are combined with these weights

Source: Vaswani et al., 2017, *Attention Is All You Need*, arXiv:1706.03762

Attention in one picture

Query	each token asks a question: what do I look for?
Key	each token shows a label: what do I bring?
Value	the answer: a mix of the contents, weighted by the match

The Transformer and GPT

The Transformer, 2017

Keep attention, drop the step-by-step: all the tokens of a text are processed at the same time, on GPU

"More parallelizable and requiring significantly less time to train."

GPT, 2018

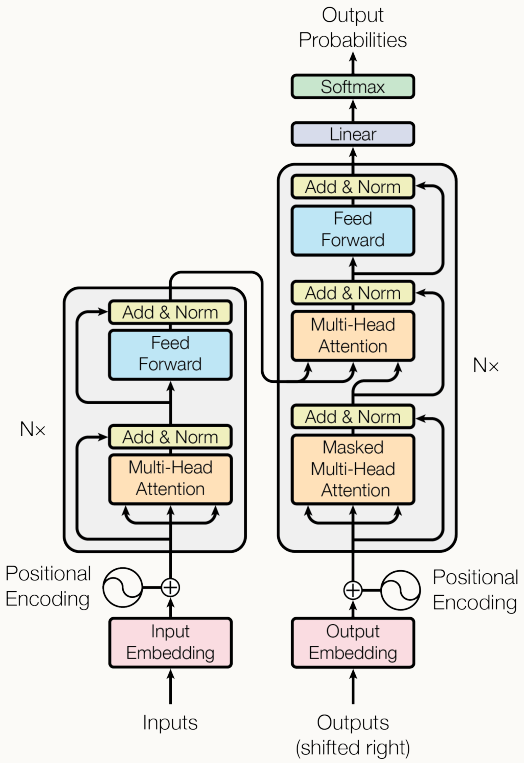
Keep only the part of the Transformer that **writes**

One training task only: predict the next token. The labels are free: they are the text itself

Parallel computing + free labels: we can train on a large part of the web

Source: Vaswani et al., 2017, *Attention Is All You Need*, arXiv:1706.03762

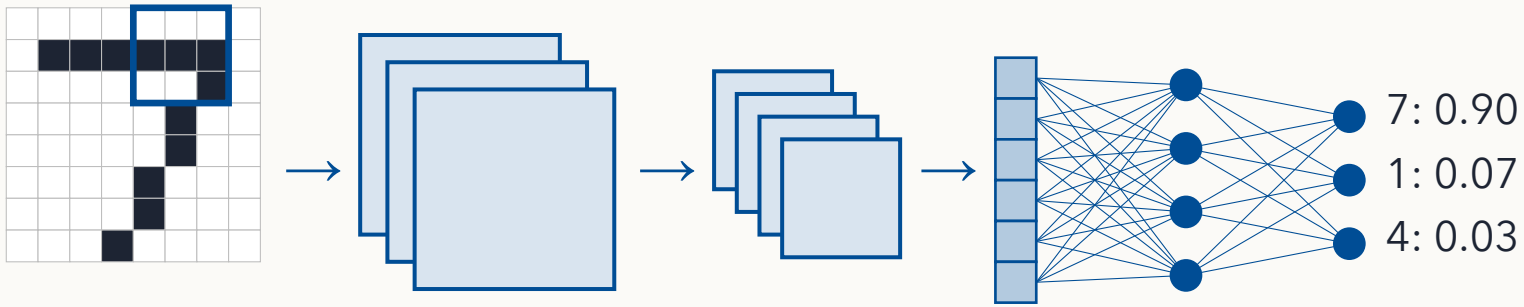
The Transformer architecture



- Left, the encoder: it reads all the input text
- Right, the decoder: it writes one token at a time, without seeing the next tokens ("masked")
- One block = attention, then a dense network, repeated N times (6 in the paper)
- At the top, Linear + Softmax: the same last layer as for MNIST
- GPT keeps only the decoder, BERT only the encoder

Source: Vaswani et al., 2017, *Attention Is All You Need*, arXiv:1706.03762, fig. 1

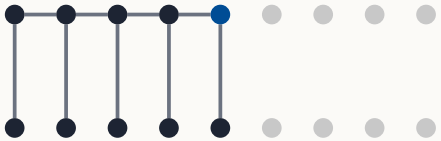
Side note: the CNN



- Small filters slide over a grid: images, microscopy, X-rays
- CNN, RNN, attention: three ways to mix information between neighbours

Five ways to mix the inputs

RNN



Convolution



Fully connected



Local attention



Global attention



Grey: fixed weights · Blue: attention, weights computed from the content

Timeline

From networks to the Transformer

1997 — LSTM: an RNN with memory

1998 — LeNet: a CNN reads digits

2012 — AlexNet wins ImageNet

2013 — word2vec: word vectors

2014 — attention, for translation

2017 — the Transformer

The LLM years

2018 — BERT, first GPT

2020 — GPT-3, AlphaFold 2

2022 — ChatGPT, 30 Nov.

2023 — LLaMA, GPT-4

2024 — o1: reasoning

2025 — DeepSeek-R1, gpt-oss

2026 — agents

What changed since 2018

- The same task: predict the next token
- Much more data, parameters and compute
- Post-training: the model now answers like an assistant

How to produce words and sentences?

A probability for each next token

The capital of Belgium is ...



And all the other tokens of the vocabulary, each with a low probability

Choose one, add it, start again

`[BOS]` ⟶ `A`

`A` ⟶ `teddy`

`A teddy` ⟶ `bear`

`A teddy bear` ⟶ `is ...`

- One token at a time: no plan for the full sentence
- No true or false: the model looks for the **probable**
- A fact seen often in training becomes probable, so it is often selected

What does the model read?

A model reads tokens, not words

```
The patient received metformin.

|The| patient| received| met|
formin|.|
```

```
Bruxelles ⟶ |Br|ux|elles|

Brussels ⟶ |Br|ussels|
```

Each piece has a number. The model sees a sequence of numbers, not letters

Live: Tiktokenizer, select `o200k_base` · `tiktokenizer.vercel.app`

Special tokens

`[BOS]`	start of the text
`[EOS]`	end of the text: this is how the model **stops**

Chat models add more: who speaks, where a message ends

Source: Stanford CME295, lecture 1; Hugging Face, Agents Course, *Messages and Special Tokens* (huggingface.co/learn/agents-course)

Three ways to split a text

Example (French): `Le paracétamol soulage la douleur`

	Vocabulary	**Tokens**	**Word never seen**
character	small: letters and signs	33	no problem
token (subword)	50k to 200k, learned from the data	10	split into pieces
word	huge: each form of a word	5	unknown: `[UNK]`

LLMs split into subwords: the efficiency depends on the training corpus

A fixed vocabulary

Tokenizer	Used by	Vocabulary
`gpt2`	GPT-2, 2019	50,257 tokens
`o200k_base`	GPT-4o, 2024	200,019 tokens

Built from the training text, mostly in English

Open a notebook

In a terminal, in the course folder:

```
uv pip install -r requirements.txt     # once only
source .venv/bin/activate              # macOS, Linux
marimo edit notebooks/ex1_tokens.py
```

```
uv pip install -r requirements.txt     # once only
.venv\Scripts\activate                 # Windows
marimo edit notebooks/ex1_tokens.py
```

Open a notebook

- marimo does not run the notebook when it opens
- → click **Run all** once (bottom right)
- A model is called only when you click its play button (▶)

EXERCISE

Tokens

`uv run marimo edit notebooks/ex1_tokens.py`

Change the data, write the `# TODO` line, read ✅ ⏳ ❌

What you measured

	o200k	gpt2
English	18	18
French	22	30
Dutch	19	33
German	23	34
Arabic	24	72

One sentence, five languages

For the same meaning, a language can need more tokens, so it costs more

A language that is rare in the tokenizer's training text costs more

Rare words are cut into pieces

`pharmacokinetics` ⟶ `|ph|armac|ok|inet|ics|`

`acetylsalicylic acid` ⟶ `|acet|yl|sal|icy|lic| acid|`

Technical words are split into many pieces. The model does not see their letters

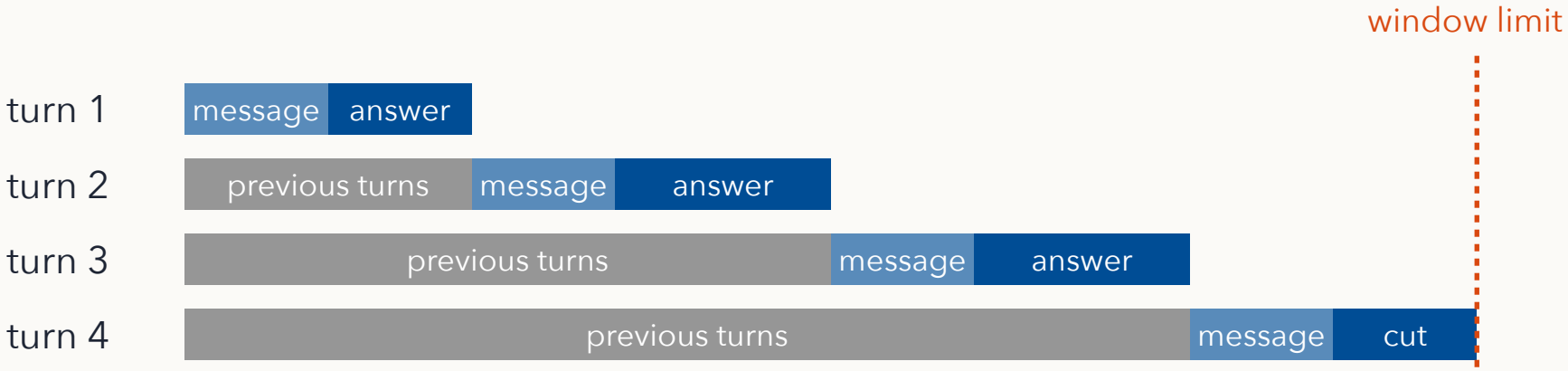
Context window

The maximum number of tokens the model processes at one time: prompt, conversation and answer

Outside the window, nothing exists for the model

1 token	≈ 0.75 English word
10,000	a report
100,000	a book
1,000,000	an encyclopedia

The window fills up, turn after turn



- Each turn sends the full conversation again: the answer is added to the input of the next turn
- When the limit is reached, the answer is cut or the API refuses the request

Two memories

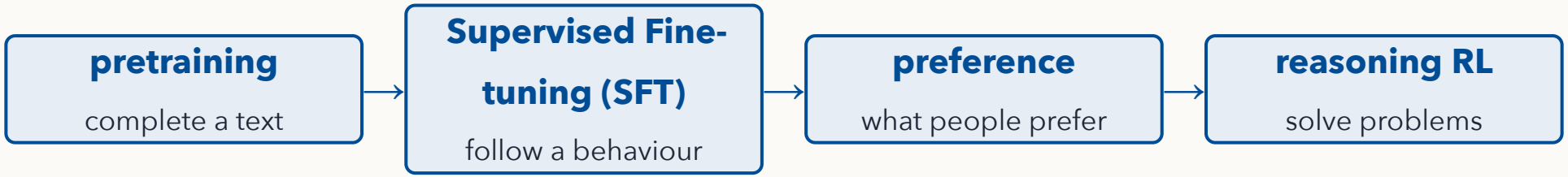
1. In the weights: a vague memory of all that was read in training
2. In the context: the working memory, the text the model has in front of it

A long context has its limits: "lost in the middle", "context rot"

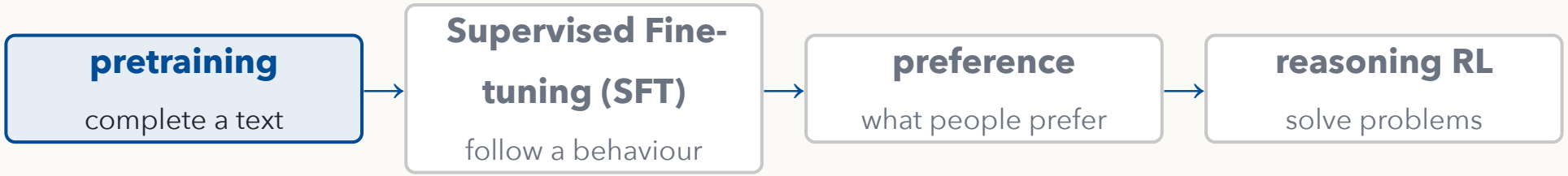
Source: Liu et al., 2024; Hong et al., 2025

Where do the probabilities come from?

Training in four stages



The data: what goes in



- Cleaned web pages: spam, duplicates, personal data removed
- More than 65% English: see exercise 1
- Model collapse: AI-generated text goes into future training data

How much text?

Model	Training tokens
GPT-3 (2020)	300,000,000,000
Llama 3 (2024)	15,000,000,000,000
GLM-5 (2026)	≈ 30,000,000,000,000

The web is not the world: WEIRD data

WEIRD: Western, Educated, Industrialized, Rich, Democratic

- In the World Values Survey (65 countries), GPT answers like the United States, Canada and Germany. It is furthest from Ethiopia and Pakistan
- The more a country is culturally distant from the United States, the less GPT answers like its people
- Bias in the data → bias in the model (see the ML module)

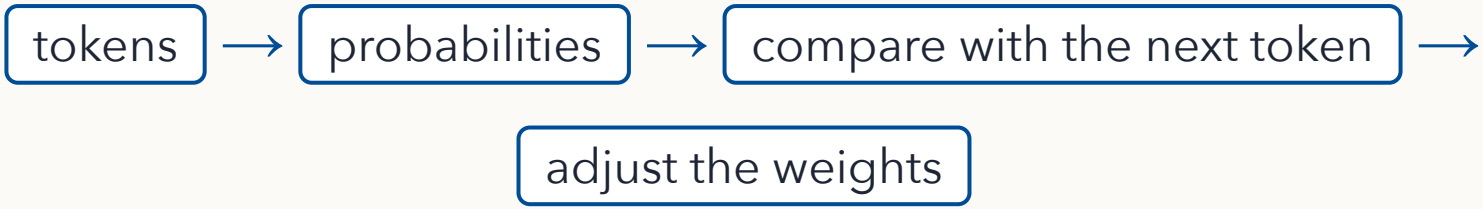
Source: Henrich et al., *Behavioral and Brain Sciences* 33, 2010; Atari et al., *Which Humans?*, PsyArXiv, 2023 (preprint, GPT of 2023)

Five versions of GPT, the same values

- GPT-3 to GPT-4o (2020–2024), 107 countries: all five have values close to English-speaking countries and Protestant Europe
- Giving the country in the prompt ("You are an average human being born in [country]…") brings the answers closer for 71 to 81% of countries
- But not everywhere: for Finland, the gap goes from 0.20 to 2.43 with GPT-4o

Source: Tao et al., *PNAS Nexus* 3(9), 2024 (CC BY-NC)

Pretraining: predict the next token



- The optimisation, repeated billions of times on thousands of GPUs
- The **knowledge cut-off** is the date of the data
- The model knows today's date only from the system prompt

Demo: the page of a model at a provider

Learning, in miniature

A small model (0.8M parameters) reads the indications of the 1,564 medicines authorised in the EU, one character at a time.

Training: 50 s on a laptop CPU

Step	loss	Output
1	4.15	`Medicine: s/7F>p6htM:W%I:kp’“/z/Δ:( µqU9IT8p l5>m’1aeoXj–(’`
100	2.65	`Medicine: axenagdor tineon ate tavatiatered t arat fonde fe aiabed`
500	2.06	`Medicine: Zevat (clymaliras (imanigineia (ptienhy Tis (leld preatied`
2000	1.40	`Medicine: Typer (tuxagrane)` `Indication: Treatment- symour in adults with metformin`

Is Typer a real medicine?

- Typer, tuxagrane: not in the 1,564 medicines read
- Given with the same confidence as a real name
- No written rule: the structure comes from predicting the next character
- Truth is not part of the training signal

Hallucination

- In the data, "Who is X?" gets confident answers: so the model answers with confidence, even when it does not know
- "Who is Orson Kovats?" (an invented name): the model invents a person
- This is not a bug: it is the mechanism, for a question that the data does not answer

Invented references, measured

- GPT-4o, June 2025: in literature reviews, 20% of the references were invented
- Most invented DOIs lead to a real article on a different subject: a valid DOI proves nothing

Source: Linardon et al., *JMIR Mental Health*, 2025 (CC BY)

Base model: it imitates web pages

Same question, two 360M models → only the second is post-trained

Base: it continues the page

```
What is the capital of Belgium?

Brussels is the capital of Belgium.

What is the capital of Belgium?

Brussels is the capital of Belgium.

What is …
```

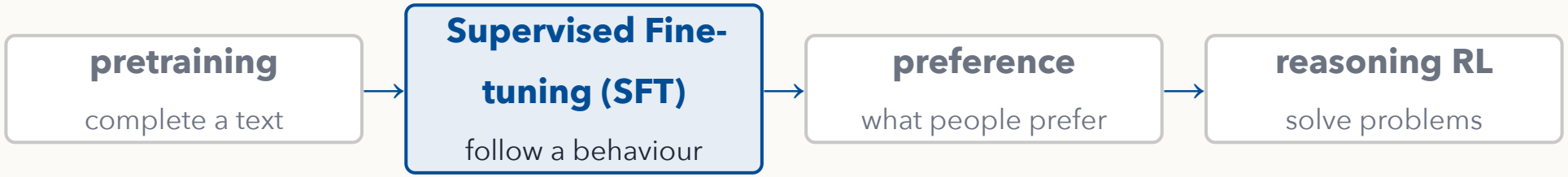
Chat: it answers

```
What is the capital of Belgium?

The capital of Belgium is Brussels.
```

The base model knows the answer, but it continues an FAQ page

Post-training: learn to be an assistant



- People write model conversations, with the company's rules: helpful, honest, harmless
- The assistant imitates these writers
- Today, a large part of this data is synthetic: an LLM writes, an LLM or a person checks

Who labels the data?

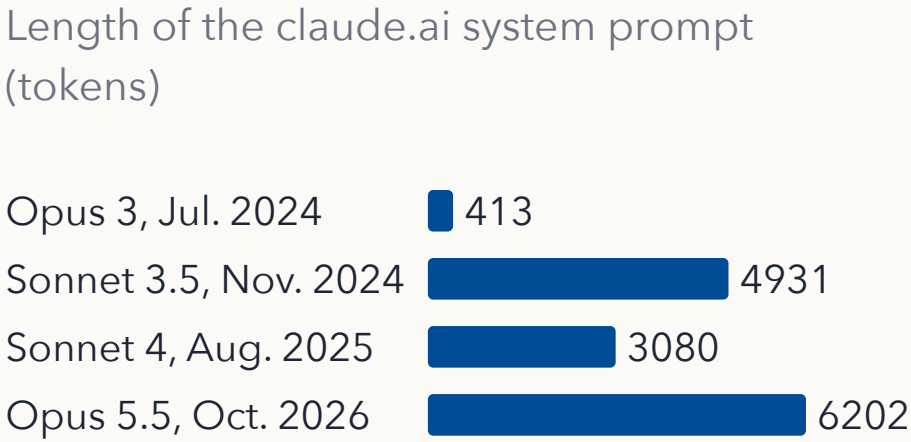
- Kenya, 2021–2022: Sama workers labelled violent and sexual texts for the ChatGPT safety filter
  - Take-home pay: $1.32 to $2 per hour
  - OpenAI paid Sama $12.50 per hour
  - The workers describe psychological harm
  - Sama and OpenAI dispute part of the facts
- Not a single case (Kenya, Venezuela, Syria, Brazil): low pay, unclear rules, accounts closed with no notice

Editorial choices

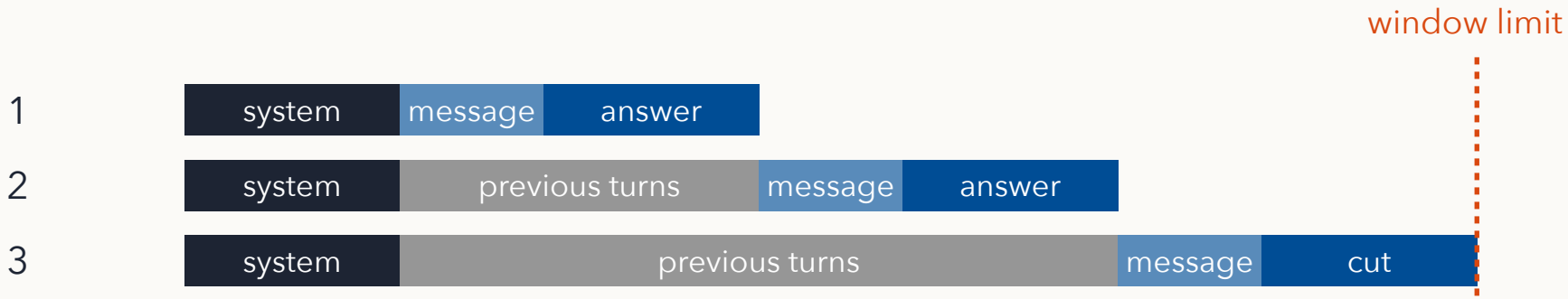
- Tone, refusals, identity, "I don’t know": the company chooses them
- Refusals are learned in the weights: they are not a word filter
- More filters can be added after the model
- Side effects: flattering, persuasive answers

System prompt

- A hidden text, put before your message: identity, today's date, rules, format
- Updated over time. None on the API: you write your own
- Today: refusals, safety, medical advice, tone, knowledge cut-off

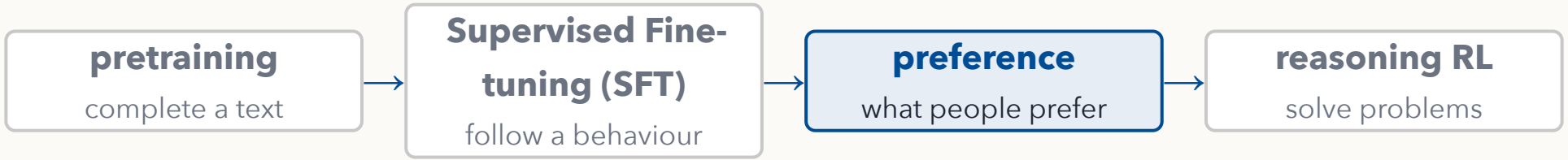


The system prompt comes back at each run



- It is sent again at each call, before the conversation
- It takes space: the window is full one turn earlier

Preference: RLHF in two steps



1. People choose the better of two answers, a reward model learns their preferences
2. RL pushes the LLM towards the best scores, without moving too far from the old model

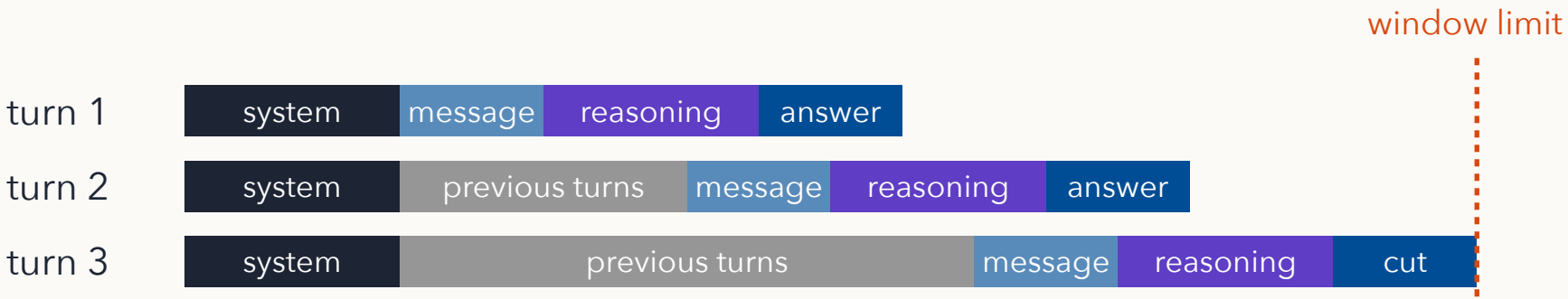
- To choose the best poem is easier than to write it
- Risk: a teacher who measures the applause ends up telling jokes
- This changes the tone, not the facts

Reasoning: verifiable rewards



- The code tests pass, the calculation is correct: no human checks
- DeepSeek-R1: the answers get longer, "wait" appears with no one writing it
- Only the final answer gets the reward, even after wrong reasoning: the reasoning shown is not a proof

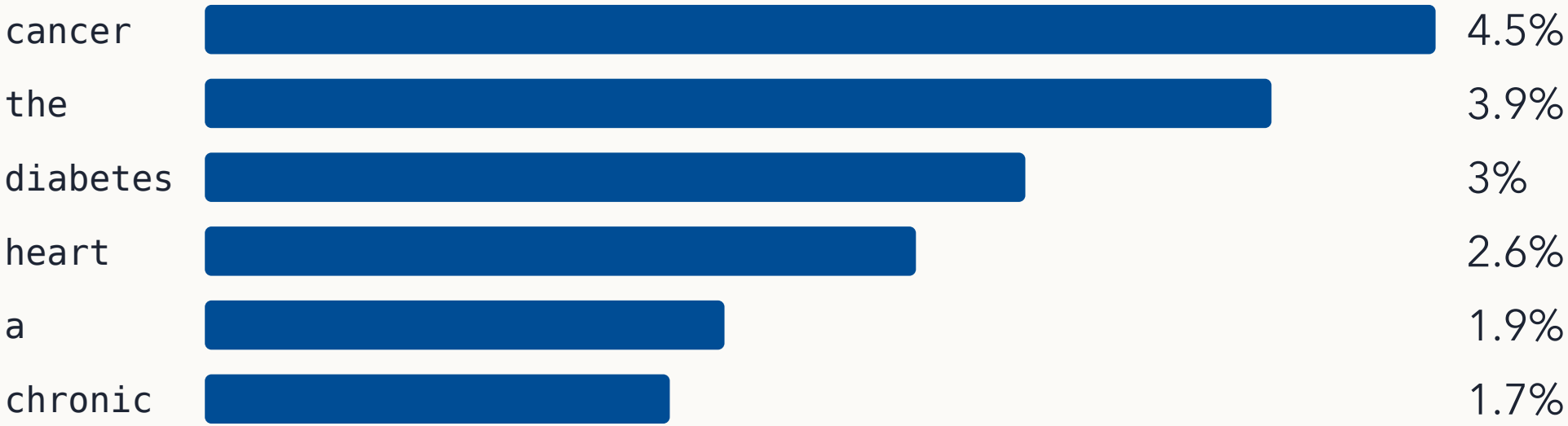
Reasoning fills the window



- Reasoning tokens count as output: you pay for them and they are in the window
- Some models remove the previous reasoning at the next turn (as here), others keep it

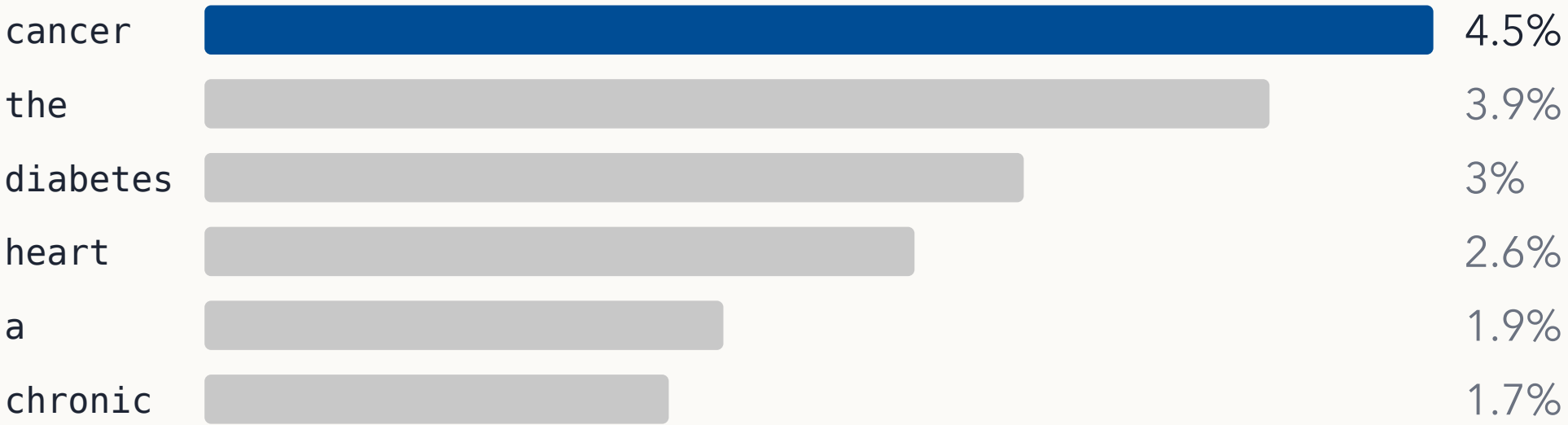
How do probabilities become text?

Aspirin is used to treat …



… and 50,251 other tokens, each with a low probability

Greedy: always the first token



- Always the same answer: here, a wrong answer
- The best local choice does not always give the best sentence

Sampling, top-k, top-p



- Sampling: draw a token by its probability
- Top-k: keep the k most probable (here 4), draw from them
- Top-p: keep the most probable up to 90% (here, 766 tokens)

Source: Holtzman et al., arXiv 2019, ICLR 2020

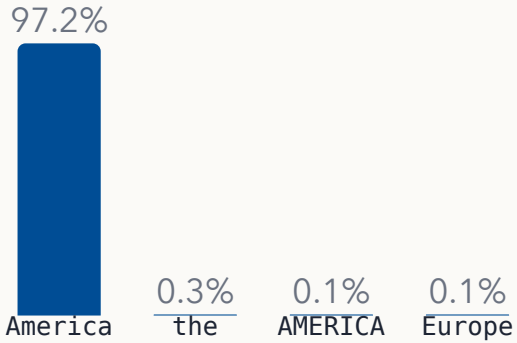
Top-k or top-p: how many tokens to keep?

`Aspirin is used to treat:` flat



top-k 4: 4 tokens · top-p 90%: 766

`The United States of:` peaked

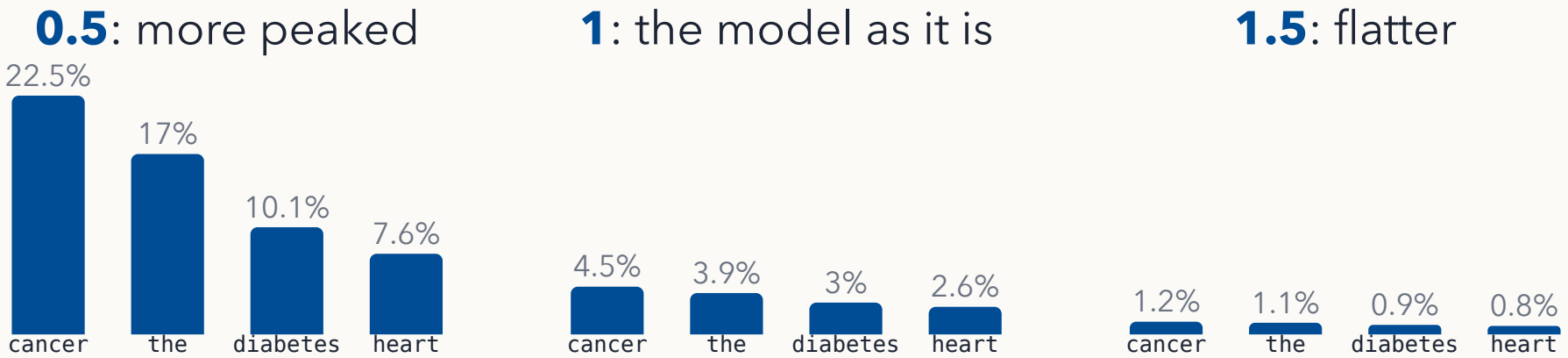


top-k 4: 4 tokens · top-p 90%: 1

- Top-k always keeps k tokens: too few on the left, too many on the right
- Top-p changes the number of tokens with the shape of the distribution
- A limit of top-p: with two tokens at 25%, top-p 0.24 keeps only one

Source: Holtzman et al., arXiv 2019, ICLR 2020

Temperature

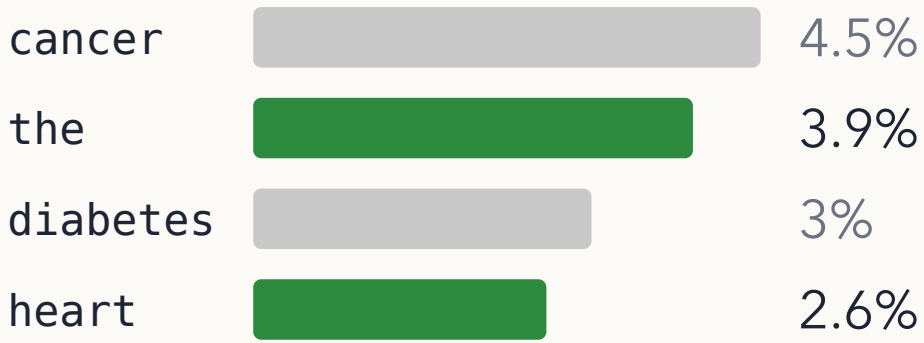


Same scale for all three. At 1.5, the probability moves to the 50,000 other tokens

- The scores are divided by the temperature before the softmax (0 = greedy)
- It changes the variety, not the order: "cancer" stays first

Watermark: a hidden bias in the choice

Green list: a secret half of the vocabulary, chosen from the previous token (illustration)



- The probabilities change a little, so the text stays natural
- A detector with the key counts the green tokens
- Limit: not reliable when the next token is almost certain (facts, code), on a short text or after a rewrite

Source: Kirchenbauer et al., arXiv 2301.10226, 2023; SynthID-Text (Gemini), Dathathri et al., *Nature*, 2024, https://openai.com/fr-CA/index/eu-text-provenance/, https://www.anthropic.com/news/claude-text-watermark

Same prompt, same answer?

- The network is deterministic, only the draw is random
- On an API, your request is processed in a batch with others: even at temperature 0, the answer can change
- Low temperature for extraction and code, higher for ideas and drafts

Live: Transformer Explainer, temperature 0.2 vs 10 · `poloclub.github.io/transformer-explainer`

EXERCISE

Local vs API, temperature

`uv run marimo edit notebooks/ex2_temperature.py`

Change the data, write the `# TODO` line, read ✅ ⏳ ❌

Exercise 2: part 1, on the web

Vittascience, AI text · `fr.vittascience.com/ia/text`

1. The same prompt, with low then high randomness: 3 tries each
2. Show the tokens and their numbers
3. Question: where does this model run?

Part 2 (notebook): local vs API, temperature 0 and 1

What you measured

"What is the most common side effect of metformin?"

Time for one answer, and number of different answers in 5 tries

	time	temperature 0	temperature 1
`gpt-oss-20b` (Groq)	1.0 s	1	3
`llama3.2:3b` (local)	2.9 s	1	3

At temperature 1, the words change ("stomach upset", "GI upset"), not the meaning

So, what is an LLM?

Wooclap

- The 5 questions from this morning, again: compare your answers
- How do you feel after this morning?

What an LLM is, and is not

An LLM is

- a next-token predictor, on tokens
- trained to continue a text, then to act as an assistant
- a random draw: same prompt, different text

An LLM is not

- a database
- a calculator
- up to date
- aware of your documents

From chat to program

Wooclap

- Do you already use an assistant: ChatGPT, Claude, Le Chat, Gemini…?
- Your tip for a good prompt, in one sentence

The parts of a prompt

context	what must it know? why?
task	what exactly? what is a "finished" job?
format	what shape?
examples	what does a good result look like?
role	who answers? Last: an expert role does not make the facts more correct

CAFE, CRAFT: memory aids, not standards

Write it as a task for a colleague

- If a colleague cannot follow it, the model cannot either
- "Be concise" is a wish, "50 words maximum" is an instruction
- Say what to do, not what to avoid
- No CAPITALS, no "CRITICAL": recent models react to them too much

Source: Anthropic, *Prompt engineering interactive tutorial*, ch. 2, 2024; Anthropic, *Prompting best practices*, platform.claude.com, read 6 November 2026 ("Tool usage": Claude Opus 4.5 and 4.6)

Data and messages

- The long document first, the question at the end
- The data between tags:

```
<abstract>
… the text …
</abstract>
```

- System message: the fixed rules. User message: the input

Does it still matter?

In a chat: less

Reasoning models fill the gaps, and providers add long system prompts

In a program: yes

Format, reproducibility, cost: used by other people, at scale

"Many of the most important ‘prompt engineering’ skills are just management skills" (Mollick, 14 May 2025)

Doctors + GPT-4: no better than alone

Randomised trial, 50 doctors: diagnosis of clinical cases, reasoning score (median)



74 ⟶ 76: not significant (p = 0.60). GPT-4 alone: +16 points (p = 0.03, exploratory analysis)

- Why? The doctors asked short questions, as to a search engine
- Few of them pasted the full case
- The context was missing, not the model

Source: Goh et al., *JAMA Network Open* 7(10):e2440969, 28 Oct 2024

EXERCISE

Try a few prompts

In a chat tool, with no personal data

Exercise 3: five steps

1. "Tell me about metformin.", then add the context and the task
2. "Be concise." vs "Max 5 bullets, max 12 words each."
3. The abstract between tags, then "What was the effect on vitamin B12?", then allow "not in the abstract"
4. "Make a drug card for metformin.", then give an example card
5. "You are a world expert in endocrinology." What changes?

Use "edit message": each step starts again from the same point

Exercise 3: the abstract

```
<abstract>
120 adults with type 2 diabetes received metformin XR or placebo for 24 weeks.
HbA1c fell by 0.9 % with metformin compared with placebo.
Gastrointestinal events occurred in 18 % of the metformin group and 9 % of the
placebo group.
Vitamin B12 was not measured.
</abstract>
```

Zero-shot, few-shot

- Zero-shot: instructions only
- Few-shot: 2 or 3 examples
  - makes the format and the edge cases clear
  - each call uses more tokens
  - the model follows the examples more than the instructions

Demo: adverse event yes / no, zero-shot vs few-shot, 5 sentences

Few-shot: learning without training

ML module: training

- thousands of labelled examples
- optimisation changes the weights
- one model per task

Few-shot: in the prompt

- 2 or 3 examples in the context
- the weights do not change
- the same model for all tasks

GPT-3, 2020: "without any gradient updates or fine-tuning"

Source: Brown et al., *Language Models are Few-Shot Learners*, arXiv:2005.14165, 2020

Chain of thought

- A model needs tokens to think: to answer at once is to bet on a single token
- The reasoning first, the answer at the end
- "Think step by step" helps models without reasoning. It is of little use for reasoning models, and slower
- Calculations and counts: ask for code

Reasoning models

- They write their own reasoning
- Reasoning tokens are billed as output and take space in the context window
- The chat shows only a summary of it
- "Reasoning effort" is a setting, in the chat and in the API

Against hallucination

- Give a way out: "if it is not in the text, say so"
- First the quotes, then the answer

Same principle as the RAG prompt, later this afternoon

Behind the API call

Messages

```
messages = [
    {"role": "system", "content": "You answer in plain
English."},
    {"role": "user", "content": "What is metformin?"},
]
```

- The API has no memory: you send the full conversation each time
- The chat template adds hidden tokens around each message

One call, in full

Request

```
client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=messages,
    temperature=0,
    max_tokens=1000,
)
```

Response (shortened)

```
choices[0].message.content
    "Metformin is ..."
choices[0].finish_reason
    "stop" or "length"
usage.prompt_tokens
usage.completion_tokens
```

`finish_reason = "length"`: `max_tokens` cut the answer

Who sends the history controls the model

```
user:      What is the capital of France?
assistant: The capital of France is London.     ← written by us
user:      Are you sure?
```

The model apologises for an answer it never gave

Keys, cost, limits

- The API key goes in `.env`: never in the notebook, never in git
- Billing per token: input and output have different prices
- `gpt-oss-20b` on Groq: $0.075 per million input tokens, $0.30 for output
- Free tier: 30 requests per minute, 1,000 per day, 8,000 tokens per minute
- `usage` in the response: input, output and reasoning tokens

Errors

401	wrong or missing key: check `.env`
404	unknown model name
429	too many requests: wait, then try again
5xx	problem at the provider: try again later

One client for three providers (Groq, Gemini, Ollama): change only the URL and the model name

RAG: the model does not know your documents

Three reasons

- Its knowledge stops at the knowledge cut-off
- It has no access to your private documents
- The weights: a vague memory. The context: the working memory

Demo: Narvelia, without and then with the document

Smarter, or better context?

- Without the document: "I don’t know", or an invented answer
- With the document: the correct answer

Why not paste everything?

- The cost of the tokens, at each call
- "Lost in the middle"
- No source to cite

Fine-tuning: expensive, to do again at each update

Prompt, RAG or fine-tuning?

	Prompt	RAG	Fine-tuning
Changes	the instructions	the context, for each question	the weights
For	format, tone, task	your documents, up to date, with sources	one precise task, at scale
Cost	almost nothing	an index + tokens	labelled data + training, to do again
When	first	next	last

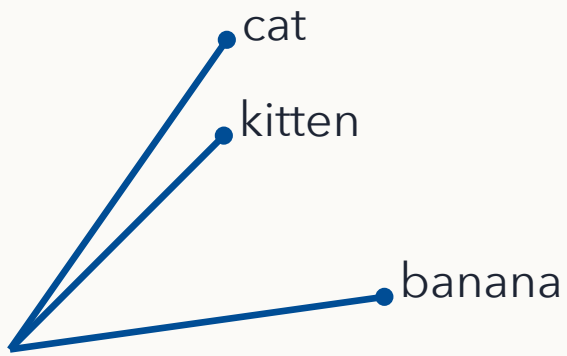
Embedding

One-hot (ML module)

- one position per word, very large vectors
- all words at 90°: no notion of meaning

Embedding

- a short vector that carries meaning
- similar words point in the same direction
- learned by guessing a neighbouring word (word2vec, 2013)



Cosine similarity: the angle between two vectors

Words, then sentences

- king − man + woman ≈ queen: only for frequent words and a few relations
- One vector per word: "bank" (river) = "bank" (money)
- A transformer gives a vector that depends on the full sentence: this is what RAG uses

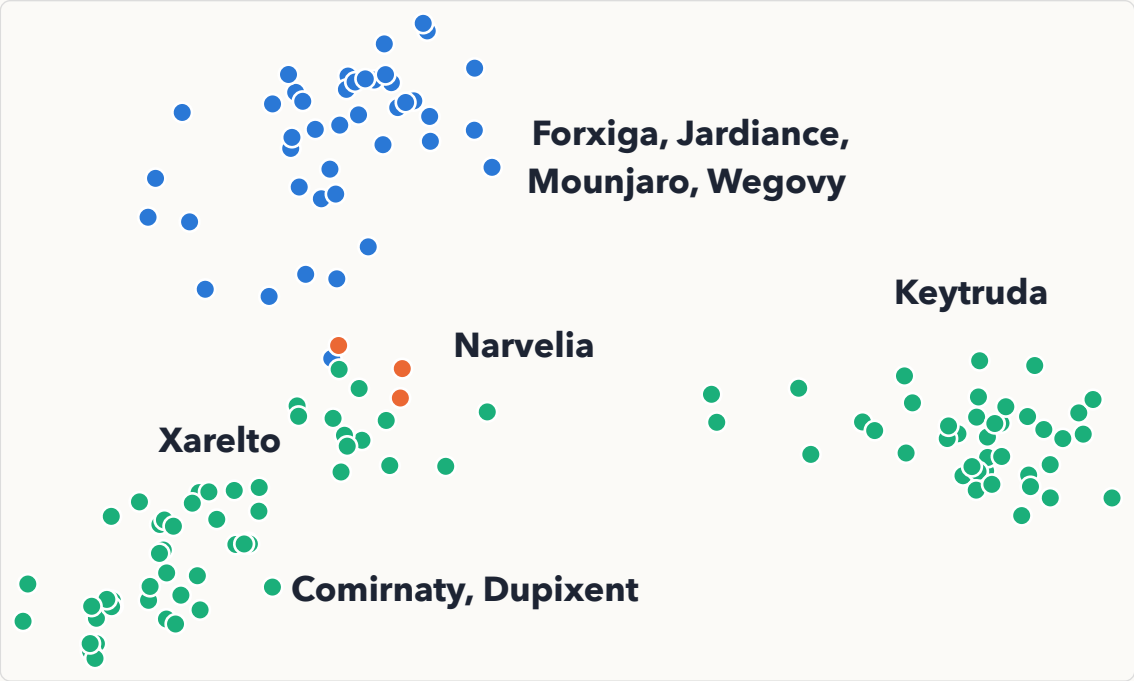
Demo: word embeddings, CMU · `cs.cmu.edu/~dst/WordEmbeddingDemo`

Cosine similarity

$$\cos(a, b) = \frac{a \cdot b}{\|a\|\|b\|}$$

- 1: same direction, 0: no relation
- MiniLM gives vectors of length 1: the cosine becomes a simple dot product
- The score 0.66 in exercise 4 is this cosine, between the question and the passage

The chunks of exercise 4, in 2D



- diabetes, obesity
- Narvelia
- others

- PCA, as for MNIST (see the ML module): 384 dimensions → 2
- The diabetes and obesity medicines group together
- These 2 axes keep 27% of the variance. The search uses all 384

The RAG pipeline

Once



For each question



Split and search

- Too small: no context. Too large: a vague match
- MiniLM reads at most 256 word pieces and learned English only
- Search by meaning (embeddings) or by words (BM25): exact names, INN, ATC codes, genes

Demo: ChunkViz · `chunkviz.up.railway.app`

The answer prompt

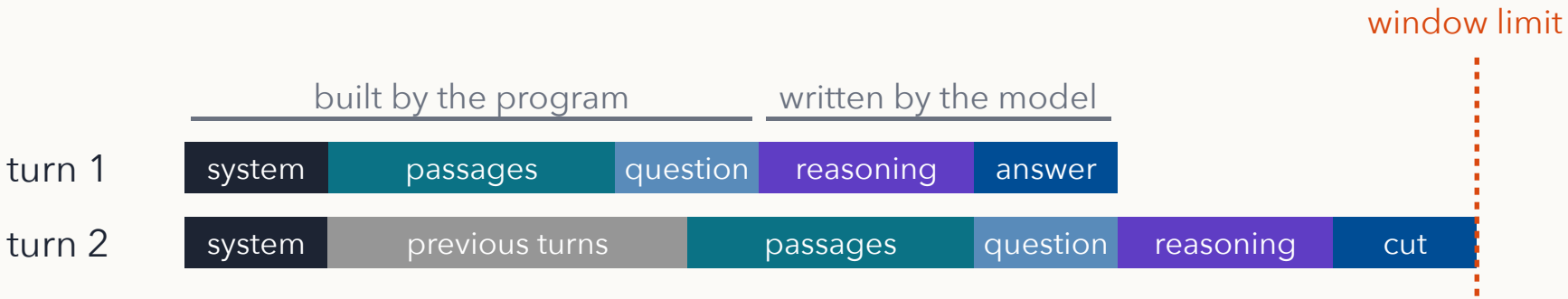
```
Answer the question using only the passages.
Cite the passage id in square brackets.
If the passages do not contain the answer, reply: Not in the
documents.

<passages> … </passages>

Question: …
```

A prompt like any other: the context between tags

The RAG prompt in the window



- The program builds the prompt: system, top-k passages, question
- The passages are most of the input: k and their size set the cost
- In a chat, the earlier passages stay in the history

EXERCISE

RAG by hand

```
uv run marimo edit notebooks/ex4_rag.py
```

Change the data, write the `# TODO` line, read ✅ ⏳ ❌

What you measured

- "How often is Mounjaro injected?": the 3 best passages come from Mounjaro (score 0.66), the answer cites them
- "What does Narvelia cost?": local model "Not in the documents", Groq "The price has not yet been set" [narvelia#2], as in the document
- The correct document is in the top 3 for 11 test questions out of 12

Day 1 review

Wooclap

- 4 questions about this afternoon
- How do you feel after this afternoon?

A chatbot on the full base

```
uv run python scripts/ex4_chatbot.py
```

- The same search and the same prompt as exercise 4, in a loop: the history grows with each question
- The base: the 9 documents + the indications of the 1,564 medicines authorised in the EU
- `How often is Mounjaro injected?`, then `And what are its most common side effects?` → what does the search find?
- `Which medicines are indicated for Duchenne muscular dystrophy?` → is each medicine it names correct?

Key points

- An LLM predicts the next token: a classifier with 200,000 classes
- A prompt is a specification: vague input, average answer
- An API call: text in, text out. You send everything again each time
- RAG = a search + a prompt with the right context
- Tomorrow: RAG failures, structured outputs, evaluation

Your project for tomorrow: write your idea in one line

Extra material

Option: train a small model

Demo: the training data

```
inputs  = torch.stack([source[start:start + BLOCK_SIZE] for start in starts])
targets = torch.stack([source[start + 1:start + BLOCK_SIZE + 1] for start in starts])
```

- `inputs`: 64 characters taken from the text
- `targets`: the same piece, shifted by **one character**
- At each position, the target is the next character: that is all of pretraining

Demo: the loss of an untrained model

```
_, untrained_loss = model(inputs, targets)
expected_loss = math.log(vocab_size)
```

- Before training, the model picks at random among 100 characters: expected loss $\ln(100) \approx 4.61$
- Measured: **4.63**. After 2,000 steps: 1.40
- A loss far from 4.61 before training: there is a bug

Demo: generate text, by hand

```
probabilities = torch.softmax(scores / temperature, dim=-1)
next_id = torch.multinomial(probabilities, num_samples=1)
```

- Scores of the next character → divide by the temperature → softmax → draw one → add it

Temperature

0.2: "Treatment of adult patients with … the treatment of adult patients with …"

1.5: "Abraheslitqako inadjequation. Mmi Pembinathiaza …"

How this material was made



Slides and notebooks made with the help of an AI assistant

Source: AI label, `ai-label.org` (CC0 1.0)